

Brief Build Document: Echidra Behavioral Classifier

Summary

The Echidra Behavioral Classifier will be built as a Python-based, explainable, rule-driven intelligence layer. It will classify attacker sessions from SSH, Telnet, FTP, and HTTP honeypot services.

The first version will focus on bot-versus-human classification using inter-keystroke timing as the primary signal, supported by authentication behavior, command patterns, protocol activity, and risk indicators.

SIEM log classification is not part of this module. The classifier accepts normalized honeypot sessions only.

Runtime modes

Mode	Purpose	Trigger
Real-time classifier	Immediate alerts and adaptive deception decisions	Every 30 seconds during active sessions
Post-session classifier	Full intelligence, evidence, reporting, and dataset labeling	After session close

Build pipeline

```
raw honeypot events
  → canonical session schema
  → feature extraction
  → feature summary generation
  → YAML rule evaluation
  → actor vote aggregation
  → behavior stage and intent mapping
  → MITRE mapping
  → risk scoring
  → evidence generation
  → deception, alert, and analyst action generation
  → API response and database storage
```

Implementation phases

Phase 1: Foundation

Create the Python package, FastAPI skeleton, tests, linting, sample data, and basic project configuration.

Phase 2: Schemas and feature extraction

Implement Pydantic schemas for normalized sessions and classifier outputs. Build feature extraction for:

- Keystroke timing.
- Authentication attempts.
- Commands.
- Protocol-specific activity.
- File and network events.
- Dashboard-oriented `feature_summary`.

Phase 3: YAML rule engine

Implement a deterministic YAML rule engine. Rules will emit:

- Actor votes.
- Behavior stages.
- Intent tags.

- MITRE tags.
- Risk deltas.
- Evidence messages.
- Deception action contributions.
- Alert action contributions.
- Analyst recommendation snippets.

Phase 4: Scoring, status, and evidence

Aggregate matched rules into one classification object. Risk score will be clamped to 0–100. Every complete or partial classification must include plain-English evidence.

The classifier must also return safe output when it cannot classify reliably:

```
{
  "classification_status": "insufficient_data",
  "insufficient_data_reason": "session duration under 10 seconds, timing
feature unavailable, and no command or auth evidence present"
}
```

Allowed status values:

- `complete` : enough data exists for reliable classification.
- `partial` : useful evidence exists but important telemetry is missing or session is still active.
- `insufficient_data` : there is not enough data to classify safely.

Phase 5: API and persistence

Expose FastAPI endpoints for real-time and post-session classification. Store post-session classifier runs and manual labels in PostgreSQL.

Phase 6: Dataset workflow

Collect real honeypot sessions for two weeks and manually label 50–100 sessions. Use synthetic sessions only for test coverage and development.

v1 actor labels

- `automated_scanner`

- `brute_force_bot`
- `commodity_bot`
- `script_kiddie`
- `skilled_human_operator`

v1 output contract

The classifier returns one object with:

- `session_id`
- `protocol`
- `classifier_mode`
- `classification_status`
- `insufficient_data_reason`
- `actor_type`
- `actor_confidence`
- `behavior_stage`
- `intent`
- `risk_score`
- `risk_level`
- `mitre_tags`
- `evidence`
- `feature_summary`
- `deception_action`
- `alert_action`
- `analyst_recommendation`
- `rule_matches`

Required action split

Do not use one vague `recommended_action` field. The output must split actions by consumer.

```
{
  "deception_action": {
    "engagement_level": "full",
    "fake_vulnerability": true,
    "decoy_files": ["/etc/shadow", "/root/.ssh/id_rsa"],
    "slow_responses": false
  },
  "alert_action": {
    "severity": 4,
    "notify": ["L2"],
    "immediate": true
  },
  "analyst_recommendation": "Monitor session actively. Check if source IP appears in production logs."
}
```

Required feature summary

`feature_summary` is mandatory because it feeds dashboard visualizations and analyst triage.

```
{
  "avg_keystroke_interval_ms": 2400,
  "login_attempts": 3,
  "unique_usernames_tried": 1,
  "commands_executed": 14,
  "payload_download_detected": false,
  "session_duration_seconds": 847,
  "human_timing_score": 0.89
}
```

Definition of success

The first version is successful when:

- It classifies sample SSH, Telnet, FTP, and HTTP sessions.
- It separates obvious bots from human-like sessions using timing and behavior signals.

- It produces evidence for every complete and partial classification.
- It returns safe insufficient-data output when classification is not possible.
- Rules are editable in YAML without code changes.
- Real-time classification can run repeatedly on active sessions.
- Post-session output is stored and ready for dashboard/reporting use.